BAN HỌC TẬP KHOA CÔNG NGHỆ PHẦN MỀM

CHUỖI TRAINING CUỐI HỌC KÌ 1 NĂM HỌC 2019 - 2020

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

Ban học tập

Khoa Công Nghệ Phần Mềm
Trường ĐH Công Nghệ Thông Tin
ĐHQG Hồ Chí Minh

Email / Group

bht.cnpm.uit@gmail.com
fb.com/groups/bht.cnpm.uit

Training

Nhập môn lập trình

Thời gian training: 10h ngày 8/1/2019

Phòng: Giảng đường 2

Trainer: Đoàn Ngọc Lãm- KTPM2019

Nguyễn Yến Nhi- KTPM2019

Phạm Nguyễn Minh Thắng- KTPM2019

Ngô Quang Vinh- KHTN2019

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Nội dung Training

I. Mảng

II. Chuỗi

III. Cấu trúc

IV. Con trỏ và đệ quy

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Mảng – Lý thuyết

1/ Khái niệm

- Biểu diễn một dãy các phần tử có cùng kiểu và mỗi phần tử trong mảng biểu diễn 1giá trị.
- Kích thước mảng được xác định ngay khi khai báo và không thay đổi.
- Một kiểu dữ liệu có cấu trúc có sẵn hay do người lập trình định nghĩa.
- Ngôn ngữ lập trình C luôn chỉ định một khối nhớ liên tục cho một biến kiểu mảng.

Sharing is learning

Mảng – Lý thuyết

2- Mảng một chiều

a. Khai báo

<tên kiểu><tên mảng>[số phần tử];

```
int array[10000];        char A[100];
```

<tên kiểu><tên mảng>[số phần tử] = {dãy giá trị };

```
int a[2] = {1, 3 }, b[3] = {3, 5 };
```

<tên kiểu><tên mảng>[ ] = {dãy giá trị };

```
double Data[] = {0, 0, 0, 0, 0 };
```

Tên kiểu: Kiểu dữ liệu int, float, char..

Tên mảng: một ký tự hoặc dãy dãy ký tự được đặt theo quy tắc đặt tên.

Số phần tử: số lượng các phần tử của mảng 1chiều.

Sharing is learning

Mảng – Lý thuyết

2- Mảng một chiều

b. chỉ số và cách truy xuất

- Chỉ số mảng (vị trí trong mảng) là một giá trị số nguyên.
- Chỉ số bắt đầu từ 0 đến số lượng phần tử - 1
- Số lượng các chỉ số mảng = số lượng phần tử tối đa trong mảng.

int A[5];

0	1	2	3	4
99	17	50	43	72

Tên mảng: A

Kiểu dữ liệu của từng phần tử trong mảng: int

Số phần tử tối đa trong mảng: 5 phần tử

Các chỉ số được đánh số: 0 → 4 (0, 1, 2, 3, 4)

Mảng – Lý thuyết

2- Mảng một chiều

b. chỉ số và cách truy xuất

- Truy xuất phần tử mảng thông qua chỉ số:

  <Tên biến mảng>[<chỉ số mảng>]

- Các phần tử mảng là 1dãy liên tục có chỉ số từ 0 đến <số phần tử mảng>- 1

Mảng – Lý thuyết

2- Mảng một chiều

b. chỉ số và cách truy xuất

int A[4]

0	1	2	3
29	137	50	4

Các truy xuất hợp lệ: A[0], A[1], A[2], A[3]

Các truy xuất không hợp lệ: A[-1], A[4], A[5]

Giá trị các phần tử mảng A[0]=29, A[1]=137, A[2]=50, A[3]=4

Sharing is learning

Mảng – Lý thuyết

2- Mảng một chiều

Truyền mảng vào hàm

- Tham số hình thức:
  - <tên kiểu> <tên hàm>(..,<tên kiểu mảng> <tên mảng>[],..)
  - <tên kiểu> <tên hàm>(..,<tên kiểu mảng> *<tên mảng>,..)
- Tham số thực:
  - <tên hàm>(.., <tên mảng>,..)

Mảng – Lý thuyết

2- Mảng một chiều

c. Cách nhập xuất mảng

Nhập mảng

```
void nhapmang(int A[], int N)
{
    for(int i=0 ; i<N; i++)
    {
        cout << "Nhap phan tu thu: " << i;
        cin >> A[i];
    }
}
```

Mảng – Lý thuyết

2- Mảng một chiều

c. Cách nhập xuất mảng

Xuất mảng

```
void xuatmang(int A[], int N)
{
    for(int i=0 ; i<N; i++)
        cout << A[i];
}
```

Mảng – Lý thuyết

2- Mảng một chiều

```
//Thuật toán sắp xếp
void swap(int &a, int &b){
    int temp= a;
    a = b;
    b = temp;
}
void exchangeSort(int a[], int n)
{
    for (int i = 0 ; i < n - 1; i++)
        for (int j = i + 1; j < n; j++)
            if (a[i] > a[j])
                swap(a[i], a[j]);
}
```

```
//Thuật toán đếm SL số nguyên tố trong mảng
bool check(int n) {
    if (n <= 1) return false;
    for (int i = 2; i <= sqrt(n); i++)
        if (n % i == 0 ) return false;
    return true;
}
int dem (int a[], int n) {
    int count = 0 ;
    for (int i = 0 ; i < n; i++) {
            if (check(a[i])) count++;
    }
    return count;
}
```

Mảng – Lý thuyết

2- Mảng một chiều

```
//thuật toán chèn phần tử x vào vị trí k của mảng (mảng có nhiều hơn 1phần tử trống).
void Insert(int a[], int &n, int x, int k)
{
    if (k<0 || k>= n) return;
    for (int i = n- 1; i >= k ; i- - ) {
        a[i + 1] = a[i];
    }
    a[k] = x; n++;
}
```

```
//thuật toán tách phần tử thứ k của mảng
void Remove(int a[], int &n, int k)
{
    if (k<0 || k>=n) return;
    for (int i = k; i <n- 1; i++)
    {
        a[i] = a[i + 1];
    }
    n- - ;
}
```

Mảng – Lý thuyết

3- Mảng hai chiều

a. Khai báo

<tên kiểu><tên mảng>**[<số hàng>][<số cột>]** ;

Ví dụ: int a[2018][2019];

float b[10][10];

Mảng – Lý thuyết

3- Mảng hai chiều

b. Chỉ số và truy vấn

	0	1	2	3
0	5	3	3	6
1	2	2	2	8
2	6	6	4	2
3	8	4	5	0

int A[4][4]; //như hình bên

Truy xuất **không** hợp lệ: a[0][2], A[5,2], A[2;3], A[- 1][2]…

Truy xuất hợp lệ: A[2][3],A[1][3]...

Giá trị của các phần tử: A[0][0]=5, A[0][1]=3,...

Các thao tác hoàn toàn giống mảng 1chiều, chỉ khác nếu ở mảng 1chiều chỉ cần sử dụng 1vòng lặp để duyệt thì mảng 2 chiều sẽ sử dụng 2 vòng lặp để duyệt.

Sharing is learning

Mảng – Bài tập

Bài tập 1: Cho một dãy số gồm n ($0 < n < 1000$) số nguyên. Viết thuật toán kiểm tra xem dãy con tăng dần liên tiếp có độ dài bằng bao nhiêu?

Ví dụ:

N=16

9 2 3 4 5 18 5 6 7 2 3 4 5 2 3

=> output = 4

Bài tập 2: Nhập vào một dãy gồm m số 1, n số 0 đại diện cho nam và nữ (1–nam, 0 - nữ) ở vị trí bất kì. Tính mức chênh lệch giữa số cặp nam-nam và nữ-nữ đứng cạnh nhau (xem các bạn nam và nữ đứng thành vòng tròn)

Ví dụ:

4 4

10 0 10 110

=> output: 0

Giải thích: có 1 cặp (1-1) và 1 cặp (0-0) đứng cạnh nhau nên đáp số là $|1-1|=0$

Mảng – Bài tập

Lời giải: Bài tập 1

```
#include <iostream>
using namespace std;
int main()
{
    int a[1000],n,d,t=1,i;
    cin>>n;
    for(int i=0;i<n;i++)
    {
        cin>>a[i];
    }
    i=0;
    while(i<n)
    {
        int j;
        j=i+1;
        d=1;
        while (a[j]>=a[j-1]&&j<n)
        {
            d++;
            j++;
        }
        t=max(d,t);
        i=j;
    }
    cout<<t;
}
```

Mảng – Bài tập

Lời giải: Bài tập 2

```
#include<bits/stdc++.h>
using namespace std;
int main(){
    int m,n;
    cin>>m>>n;

    int a[200000];
    for (int i=0;i<m+n;i++)
        cin>>a[i];
    int nam=0,nu=0;
    //biến đếm số cặp nam nữ đứng cạnh nhau
    a[m+n]=a[0];
    //xem như mảng đã thành vòng tròn
    for(int i=0;i<m+n;i++)
    {
        if(a[i]==a[i+1]&&a[i]==0) nu++;
        if(a[i]==a[i+1]&&a[i]==1) nam++;
    }
    cout<<abs(nam-nu);
}
```

Mảng – Bài tập

Bài tập 1:Viết chương trình kiểm tra một ma trận m hàng , n cột có phải là ma trận đối xứng qua trục dọc hay không? Nếu có xuất ra **TRUE** còn không xuất ra **FALSE**.

Ví dụ:

2 5

12042 30563 24105 30563 12042

25363 30304 18436 30304 25363

=> output =TRUE

Bài tập 2: Tìm vị trí của những phần tử yên ngựa trong 1 ma trận mxn. Phần tử yên ngựa và phần tử có giá trị lớn hơn hẳn các vị trí xung quanh nó

Ví dụ:

3 3

1 2 3

4 9 5

6 7 8

=> Output: 1 1

Mảng – Bài tập

Lời giải: Bài tập 1

```
#include <iostream>
using namespace std;
const int N = 1000;
bool check(int a[][N], int m, int n)
{
    for (int i=0;i<m;i++)
    {
        int j=0;
        while (j<n/2)
        {
            if (a[i][j]!=a[i][n-j-1]) return( false);
            else j++;
        }
    }
    return (true);
}
int main()
{
    int a[N][N],m,n;
    cin >> m >> n;
    for (int i=0; i < m; i++)
    for (int j=0; j < n; j++)
        cin >> a[i][j];
    if (check(a,m,n)) cout<<"TRUE";
    else cout<<"FALSE";
}
```

Mảng – Bài tập

Lời giải: Bài tập 2

```
#include <iostream>
using namespace std;
int main()
{
    int a[1000][1000],b[1000][1000],m,n,i,j;
    long long sum;
    cin>>n>>m;
    for (i=0;i<n;i++)
        for (j=0;j<m;j++)
            cin>>a[i][j];
    for (i=1;i<n-1;i++)
        for (j=1;j<m-1;j++)
        {
            if (a[i][j]>a[i][j-1]&&a[i][j]>a[i-1][j]
                &&a[i][j]>a[i+1][j]&&a[i][j]>a[i][j+1]
                &&a[i][j]>a[i-1][j-1]&&a[i][j]>a[i-1][j+1]
                &&a[i][j]>a[i+1][j-1]&&a[i][j]>a[i+1][j+1])
            cout<<i<<" "<<j<<endl;
        }
}
```

Chuỗi – Khái niệm

❖ Khái niệm

- Kiểu char chỉ chứa được một ký tự. Để lưu trữ một chuỗi (nhiều ký tự) ta sử dụng mảng (một chiều) các ký tự.
- Chuỗi ký tự kết thúc bằng ký tự " \0 "(null)

=>Độ dài chuỗi = kích thước mảng – 1

❖ Ví dụ

```
char    szHoten[30];        // Dài 29 ký tự
char    szNgaySinh[9];      // Dài 8 ký tự
```

Khởi tạo

- Khởi tạo như mảng thông thường
  - Độ dài cụ thể

```
char s[10] = {'T', 'H', 'C', 'S', ' ', 'A', '\0'};
char s[10] = "THCS A"; // Tự động thêm '\0'
```

0	1	2	3	4	5	6	7	8	9
'T'	'H'	'C'	'S'	' '	'A'	'\0'			

  - Tự xác định độ dài

```
char s[] = {'T', 'H', 'C', 'S', ' ', 'A', '\0'};
char s[] = "THCS A"; // Tự động thêm '\0'
```

0	1	2	3	4	5	6
'T'	'H'	'C'	'S'	' '	'A'	'\0'

Nhập chuỗi

- Sử dụng hàm scanf với đặc tả "%s"
  - Chỉ nhận các ký tự từ bàn phím đến khi gặp ký tự khoảng trắng hoặc ký tự xuống dòng.
  - Chuỗi nhận được không bao gồm ký tự khoảng trắng và xuống dòng.

```
char String[50];
printf("Nhap motchuoi: ");
scanf("%s", String);
printf("Chuoinhan duoc la: %s", String);
```

```
Nhap mot chuoi: Nhap mon lap trinh
Chuoi nhan duoc la: Nhap_
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Nhập chuỗi

- Sử dụng hàm gets
  - Nhận các ký tự từ bàn phím đến khi gặp ký tự xuống dòng.
  - Chuỗi nhận được là những gì người dùng nhập (trừ ký tự xuống dòng).

```
char String[50];
printf("Nhap motchuoi: ");
gets(String);
printf("Chuoinhan duoc la: %s", String);
```

```
Nhap mot chuoi: Nhap mon lap trinh
Chuoi nhan duoc la: Nhap mon lap trinh_
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Nhập chuỗi

❖ **Lưu ý:** khi cần xóa bộ nhớ đệm dùng fflush(stdin) hoặc cin.ignore()

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main()
6   {
7       int a;
8       char s[30];
9       scanf("%d",&a);
10      gets(s);
11      printf("%d\n", a);
12      printf("%s",s);
13      return 0;
14  }
15
```

```
1   #include <iostream>
2
3   using namespace std;
4
5   int main()
6   {
7       int a;
8       char s[30];
9       scanf("%d",&a);
10      cin.ignore();
11      gets(s);
12      printf("%d\n", a);
13      printf("%s",s);
14      return 0;
15  }
16
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Xuất chuỗi

- Sử dụng hàm printf với đặc tả "%s"

```
Char string[50] = "Nhap mon lap trinh";
Printf("%s", string);      // Không xuống dòng
```

```
Nhap mon lap trinh_
```

- Sử dụng hàm puts

```
Char string[50] = "Nhap mon lap trinh";
Puts(string);      // Tự động xuống dòng
```

```
Nhap mon lap trinh
_
```

Hàm chuỗi trong C

❖ Thuộc thư viện <string.h>

- strlen: đếm số phần tử có trong string không tính ký tự null (trả về int)
- strcpy: copy chuỗi này qua chuỗi khác
- strlwr/strupr: chuyển đổi chữ hoa và thường (trả về con trỏ)
- strrev: đảo ngược các kí tự trong chuỗi (trả về con trỏ)
- strcmp/stricmp: so sánh 2 chuỗi (trả về int)
- strcat: nối 2 chuỗi
- strstr: tìm kiếm chuỗi con và trích chuỗi còn lại (trả về con trỏ)

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Hàm Strlen()

Cú pháp: int strlen(const char* s)

Ví dụ: Sử dụng hàm strlen xác định độ dài một chuỗi nhập từ bàn phím.

```c
#include<conio.h>
#include<stdio.h>
#include<string.h>
void main()
{

    char Chuoi[255];
    int Dodai;
    printf("Nhap chuoi: ");
    gets(Chuoi);
    Dodai = strlen(Chuoi);
    printf("Chuoi vua nhap: ");
    puts(Chuoi);
    printf("Co do dai %d",Dodai);
    getch();
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Hàm Strcpy()

– Hàm strcpy(): Hàm này được dùng để sao chép toàn bộ nội dung của chuỗi nguồn vào chuỗi đích.

Cú pháp: char *strcpy(char *Des, const char *Source)

Ví dụ: Viết chương trình cho phép chép toàn bộ chuỗi nguồn vào chuỗi đích.

```c
#include<conio.h>
#include<stdio.h>
#include<string.h>

void main()
{

    char Chuoi[255],s[255];
    printf("Nhap chuoi: ");
    gets(Chuoi);
    strcpy(s,Chuoi);
    printf("Chuoi dich: ");
    puts(s);
    getch();
}
```

Hàm Strcat()

– Hàm strcat(): Hàm này được dùng để nối toàn bộ nội dung của chuỗi nguồn vào sau chuỗi đích.

Cú pháp: char *strcat(char *Des, const char *Source)

Ví dụ: Viết chương trình cho phép nối toàn bộ chuỗi nguồn vào sau chuỗi đích.

```
1   #include <conio.h>
2   #include <string.h>
3   #include<iostream>
4   using namespace std;
5
6   int main()
7   {
8       char Chuoi[255], s[255];
9       printf("Nhap chuoi: ");
10      gets(Chuoi);
11      printf("Nhap s: ");
12      gets(s);
13      strcat(Chuoi,s);
14      puts(Chuoi);
15  }
```

Hàm Strupr()

– Hàm strupr(): Hàm struppr() được dùng để chuyển đổi chuỗi chữ thường thành chuỗi chữ hoa, kết quả trả về của hàm là một con trỏ chỉ đến địa chỉ chuỗi được chuyển đổi.

Cú pháp: `char *strupr(char *s)`

Ví dụ:

```c
#include<conio.h>
#include<stdio.h>
#include<string.h>
void main()
{
  char Chuoi[255],*s;
  printf("Nhap chuoi: ");
  gets(Chuoi);
  s=strupr(Chuoi) ;
  printf("Chuoi chu hoa: ");
  puts(s);
  getch();
}
```

Hàm Strcmp()

- Hàm strcmp(): Để so sánh hai chuỗi theo từng ký tự trong bảng mã Ascii, ta có thể sử dụng hàm strcmp().

Cú pháp: int strcmp(const char *s1, const char *s2)

Hai chuỗi s1 và s2 được so sánh với nhau, kết quả trả về là một số nguyên (số này có được bằng cách lấy ký tự của s1 trừ ký tự của s2 tại vị trí đầu tiên xảy ra sự khác nhau).

- Nếu kết quả là số âm, chuỗi s1 nhỏ hơn chuỗi s2;
- Nếu kết quả là 0, hai chuỗi bằng nhau;
- Nếu kết quả là số dương, chuỗi s1 lớn hơn chuỗi s2.
- VD:

```
cout<<strcmp("A", "a");   // In ra "-1" do A<a
```

Hàm Strstr()

– Hàm strstr(): Hàm strstr() được sử dụng để tìm kiếm sự xuất hiện đầu tiên của chuỗi s2 trong chuỗi s1.

Cú pháp: char *strstr(const char *s1, const char *s2)

Kết quả trả về của hàm là một con trỏ chỉ đến phần tử đầu tiên của chuỗi s1 có chứa chuỗi s2 hoặc giá trị NULL nếu chuỗi s2 không có trong chuỗi s1.

Ví dụ: Viết chương trình sử dụng hàm strstr() để lấy ra một phần của chuỗi gốc bắt đầu từ chuỗi "hoc".

```
#include<conio.h>
#include<stdio.h>
#include<string.h>

void main()

{
    char Chuoi[255],*s;
    printf("Nhap chuoi: ");
    gets(Chuoi);
    s=strstr(Chuoi,"hoc");
    printf("Chuoi trich ra: ");
    puts(s);
    getch();
}
```

Hướng suy nghĩ trong xử lý chuỗi

BT1: Tim Tiếng

Viết chương trình nhập chuỗi ký tự. Nếu chuỗi nhập vào là chuỗi rỗng, chương trình in ra thông báo "Chuoi rong.". Ngược lại, chương trình đếm đếm số lần xuất hiện của từng tiếng trong chuỗi.

Input	Output
	Chuoi rong.
Hoc sinh hoc sinh hoc	Hoc: 1 sinh: 2 hoc: 2

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
haring is learning
ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Cấu trúc

1 Khái niệm, khai báo:

- Kiểu cấu trúc (struct) cho phép chúng ta định nghĩa một đối tượng có nhiều thuộc tính.

- Cách khai báo:

```
struct <tên kiểu cấu trúc>
{
    <kiểu dữ liệu> <tên thành phần 1>;
    …
    <kiểu dữ liệu> <tên thành phần n>;
};
```

Cấu trúc

1 Khái niệm, khai báo:

Ví dụ: Định nghĩa (Khai báo) cấu trúc Sinh viên (gồm MSSV, họ tên, lớp).

```
struct sinhvien
{
    char mssv[8];
    char hoten[250];
    char lop[10];
};
```

Cấu trúc

1. Khái niệm, khai báo:

- Cú pháp không tường minh:

```
struct <tên kiểu cấu trúc>
{
    <kiểu dữ liệu> <tên thành phần 1>;
    …
    <kiểu dữ liệu> <tên thành phần n>;
};
struct <tên kiểu cấu trúc> <tên biến>;
```

Cấu trúc

1. Khái niệm, khai báo:

- Cú pháp không tường minh:

- Ví dụ:

```
struct DIEM
{
    int x;
    int y;
};
struct DIEM diem1, diem2;  //C++ có thể bỏ struct
```

Cấu trúc

1 Khái niệm, khai báo:

- Sử dụng typedef

- Cú pháp:

```
typedef struct
{
    <kiểu dữ liệu> <tên thành phần 1>;
    …
    <kiểu dữ liệu> <tên thành phần n>;
} <tên kiểu cấu trúc>;
<tên kiểu cấu trúc> <tên biến>;
```

Cấu trúc

1 Khái niệm, khai báo:

- Sử dụng typedef

- Ví dụ:

```
typedef struct DIEM
{
    int x;
    int y;
};
struct DIEM diem1, diem2; //C++ có thể bỏ struct
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Cấu trúc

1. Khái niệm, khai báo:

- Cú pháp tường minh:

```
struct <tên kiểu cấu trúc>
{
   <kiểu dữ liệu> <tên thành phần 1>;
   …
   <kiểu dữ liệu> <tên thành phần n>;
} <tên biến> = {<giá trị 1>,…,<giá trị n>};
```

- Ví dụ:

```
struct DIEM
{
   int x;
   int y;
} diem1 = {2912, 1706}, diem2;
```

Cấu trúc

2. Truy xuất dữ liệu kiểu cấu trúc:

- Không thể truy xuất trực tiếp mà phải thông qua toán tử chấm (.) (dot operation).
- Cú pháp:

```
<tên kiểu cấu trúc>.<tên thành phần>
```

- Ví dụ:

```
struct SinhVien
{
    char ten[250];
    char lop[10];
} SinhVien1;

cout << SinhVien1.ten << " " << SinhVien1.lop;
```

Cấu trúc

3. Gán dữ liệu kiểu cấu trúc:

- Có 2 cách truy xuất:

```
<biến cấu trúc đích> = <biến cấu trúc nguồn>;

<biến cấu trúc đích>.<tên thành phần> = <giá trị>;
```

- Ví dụ:

```
struct DIEM
{
    int x, y;
} diem1 = {2912, 1706}, diem2;
…
diem2 = diem1;
diem2.x = diem1.x;
diem2.y = diem1.y * 2;
```

Cấu trúc

4. Cấu trúc phức tạp:

- Thành phần của cấu trúc là cấu trúc khác

```
struct DIEM
{
    int x;
    int y;
};

struct HINHCHUNHAT
{
    struct DIEM traitren;
    struct DIEM phaiduoi;
} hcn1;
…
hcn1.traitren.x = 2912;
hcn1.traitren.y = 1706;
```

Cấu trúc

4. Cấu trúc phức tạp:

- Cấu trúc đệ quy (tự trỏ)

```
struct PERSON
{
    char hoten[30];
    struct PERSON *father, *mother;
};

struct NODE
{
    int value;
    struct NODE *pNext;
};
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Cấu trúc

5. Lưu ý:

- Kiểu cấu trúc được định nghĩa để làm khuôn dạng còn biến cấu trúc được khai báo để sử dụng khuôn dạng đã định nghĩa.
- Trong C++, có thể bỏ từ khóa struct khi khai báo biến (hoặc sử dụng typedef)
- Khi nhập các biến kiểu số thực trong cấu trúc phải nhập thông qua một biến trung gian.

```
struct DIEM { float x, y;} d1;
float temp;
cin >> temp;
d1.x = temp;
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Cấu trúc

6. Mảng cấu trúc:

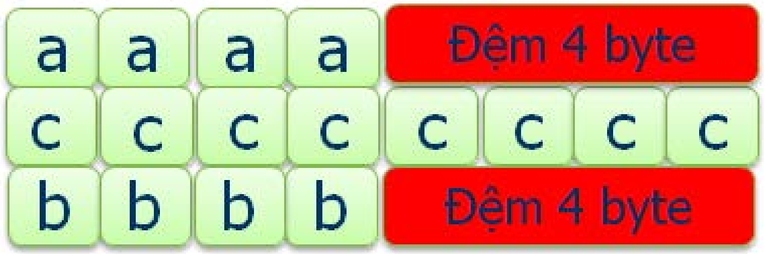
- Tương tự như mảng với kiểu dữ liệu cơ sở (char, int, float, ..)

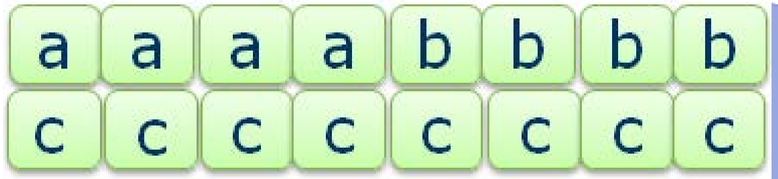
```
struct DIEM
{
    int x;
    int y;
};

DIEM mang1[20];
DIEM mang2[10] = {{3, 2}, {4, 4}, {2, 7}};
```

Cấu trúc

7. Kích thước cấu trúc:

```
struct B1
{
    int a;
    double c;
    int b;
};
sizeof (B1) = ??24
struct B2
{
    int a;
    int b;
    double c;
};
sizeof (B2) = ??16
```





Sự khác biệt đến từ thứ tự khai báo các biến và biên kích thước (tính theo byte) của cấu trúc.
Biên mặc định của VC++ là 8 byte.

Cấu trúc

7. Kích thước cấu trúc:

Tối ưu kích thước cấu trúc dựa trên thứ tự các biến (tối ưu cục bộ trên cấu trúc).

Hoặc tối ưu biên cho cấu trúc (alignment of struct). Ví dụ trên nếu thay đổi biên cấu trúc thành 1hoặc 4 thì **sizeof(B1) = 16.**

Điều chỉnh biên cấu trúc: **Project settings Compile Option C/C++ Code Generation Structure Alignment.**

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Cấu trúc

7. Kích thước cấu trúc:

Dễ dàng điều chỉnh biên để tối ưu. Hay biên cấu trúc càng nhỏ càng giúp giảm vùng đệm thì càng tốt ??

Không. Biên nhỏ giúp giảm kích thước của cấu trúc nhưng làm tăng thời gian xử lý của tác vụ memory allocator Cần điều phối thích hợp giữa kích thước cấu trúc và tốc độ xử lý.

Chương trình dùng nhiều cấu trúc có thành phần khác nhau điều chỉnh biên tốt nhất sẽ khó khăn

Ưu tiên: tối ưu bằng cách khai báo thứ tự các thành phần cấu trúc phù hợp với biên cấu trúc.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM BAN HỌC TẬP Sharing is learning

Cấu trúc

8. Truyền cấu trúc cho hàm:

- Giống như truyền kiểu dữ liệu cơ sở
  - Tham trị (không thay đổi sau khi kết thúc hàm)
  - Tham chiếu
  - Con trỏ

Ví dụ:

```
struct DIEM
{
    int x, y;
};
void xuat1(int x, int y) { … };
void xuat2(DIEM diem) { … };
void xuat3(DIEM &diem) { … };
void xuat4(DIEM *diem) { … };
```

Con trỏ

- **Khái niệm:** Một con trỏ là một biến, nó chứa địa chỉ vùng nhớ của một biến khác, chứ không lưu trữ giá trị của biến đó.
- **Khai báo:** Giống như mọi biến khác, biến con trỏ muốn sử dụng cũng cần phải được khai báo

<kiểu dữ liệu>*<tên biến con trỏ>;

Ví dụ: char *ch1, *ch2;
int *p1, p2;

Khai báo

- Sử dụng từ khóa typedef:

typedef <kiểu dữ liệu> *<tên kiểu con trỏ>;

<tên kiểu con trỏ> <tên biến con trỏ>;

Ví dụ:

```
typedef int *pint;
pint p2, p3;
```

- Lưu ý khi khai báo kiểu dữ liệu mới:

+Giảm bối rối khi mới tiếp xúc với con trỏ.

+Nhưng dễ nhầm lẫn với biến thường.

- Lưu ý về con trỏ NULL:

+Con trỏ NULL là con trỏ không trỏ vào đâu cả.

+Khác với con trỏ chưa được khởi tạo.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Con trỏ và cấp phát động

- Cách dùng:

+Cách 1: Sử dụng con trỏ để giữ địa chỉ của một biến

+Cách 2: Sử dụng con trỏ để xin cấp phát và thu hồi bộ nhớ

- Các toán tử cơ bản:

+Toán tử &: được sử dụng để lấy địa chỉ của một biến

+Toán tử *: được sử dụng để lấy và cập nhật giá trị tại địa chỉ mà biến con trỏ đang giữ.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Con trỏ và cấp phát động

- Khai báo biến con trỏ:

  <kiểu con trỏ> *<tên con trỏ>; (hoặc sử dung typedef)

- Trỏ con trỏ đến biến:

  <tên con trỏ> = & <tên biến>;

- Lấy giá trị nơi con trỏ trỏ đến:

  *<tên con trỏ>;

- Lấy địa chỉ nơi con trỏ trỏ đến:

  &<tên con trỏ>;

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Con trỏ và cấp phát động

- Các phép toán với con trỏ:

+Phép toán gán: (thực chất là trỏ con trỏ đến một biến)

Ví dụ:

```
int *p,a =10;
p = &a;
```

+Phép toán tăng giảm (++,--,+=,-=,..): (phần này sẽ trình bày cụ thể hơn ở phần *Con trỏ và mảng*)

Ví dụ:

```
int *p, a[]={0,2,4};
p=&a[0];p++;   //sau khi p++ thì p trỏ đến a[1]
```

+Phép toán so sánh giữa hai con trỏ: thực chất là so sánh địa chỉ của hai nơi được trỏ bởi hai con trỏ này.

Con trỏ và cấp phát động

- **Cấp phát và thu hồi bộ nhớ:**
  + Cấp phát 1phần tử:
  <tên con trỏ>= new <kiểu dữ liệu>;
  + Cấp phát nhiều phần tử:
  <tên con trỏ>= new <kiểu dữ liệu>[số phần tử];
  + Giải phóng bộ nhớ một biến:
  delete <tên con trỏ>;
  + Giải phóng bộ nhớ một mảng:
  delete[] <tên con trỏ>;
- **Lưu ý:** khi cấp phát bộ nhớ cho con trỏ bằng toán tử new thì khi thu hồi bộ nhớ ta bắt buộc phải dùng toán tử delete.

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Con trỏ và hàm

- Truyền địa chỉ (con trỏ):

```
#include <stdio.h>
void hoanvi(int *x, int *y);
void main()
{
    int a = 2912; b = 1706;
    hoanvi(&a, &b);
    printf("a = %d, b = %d", a, b);
}
void hoanvi(int *x, int *y)
{
    int t = *x; *x = *y; *y = t;
}
```

Con trỏ và hàm

- Truyền tham chiếu (C++):

```
#include <stdio.h>
void hoanvi(int &x, int &y);
void main()
{
    int a = 2912; b = 1706;
    hoanvi(a, b);
    printf("a = %d, b = %d", a, b);
}
void hoanvi(int &x, int &y)
{
    int t = x; x = y; y = t;
}
```

Con trỏ và mảng 1 chiều

Tên của mảng là một hằng địa chỉ, chứa địa chỉ của phần tử đầu tiên.

Với khai báo: int a[10] thì:

a tương đương với &a[0]

a+i tương đương với &a[i]

*(a+i) tương đương với a[i]

Xét khai báo : int a[5], *pa=a; khi đó con trỏ pa giữ địa chỉ của phần tử đầu tiên của mảng a. Lúc này:

pa+i tương đương với &a[i].

*(pa+i) tương đương với a[i].

Con trỏ và mảng 2 chiều

Mảng 2 chiều là mảng (1chiều) của mảng.

Khi khai báo: int a[2][3] thì trong bộ nhớ tạo ra:

Phần tử	a[0][0]	a[0][1]	a[0][2]	a[1][0]	a[1][1]	a[1][2]
Địa chỉ	1	2	3	4	5	6

Tức mảng a có 2 phần tử, mỗi phần tử là 1mảng chứa 3 số nguyên.

Tên mảng a chứa địa chỉ của phần tử đầu tiên.

Địa chỉ phần tử đầu tiên có kiểu int[3]

Với khai báo: int a[2][3] thì:

a tương đương với &a[0][0]

a+1tương đương với &a[1][0]

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Con trỏ và mảng 2 chiều

Xét khai báo : int a[2][3], *pa;

pa = (int*)a;

Khi đó:

pa tương đương với &a[0][0].

pa+1tương đương với &a[0][1].

pa+2 tương đương với &a[0][2].

pa+3 tương đương với &a[1][0].

pa+4 tương đương với &a[1][1].

pa+5 tương đương với &a[1][2].

Chú ý: vì a là địa chỉ kiểu int[3], pa là con trỏ kiểu int. Nên phải ép kiểu như sau: pa = (int*)a;

Con trỏ và mảng 2 chiều

Ví dụ về nhập mảng hai chiều bằng con trỏ:

```
#include <stdio.h>
#include <conio.h>
#define RMax 100
#define CMax 100
void main()
{
  int a[RMax][CMax],n,m, i, j, *pa;
  pa=(int*)a;
  printf("So dong va cot:");
   scanf("%d%d",&n,&m);
```

```
 for(i=0;i<n;i++)
   for (j=0;j<m;j++)
   {
     printf("Nhap phan tu a[%d][%d]: ",i,j);
     scanf("%d", pa + i*m +j);
   }
 for(i=0;i<n;i++)
 {
   for (j=0;j<m;j++)
   {
     printf("%d ", *(pa + i*m +j));
   }
   printf("\n");
 }
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Đệ quy

- **Khái niệm:** Một hàm được gọi là đệ quy nếu bên trong thân của hàm có lời gọi hàm lại chính nó một cách tường minh hay tìm ẩn.

- **Phân loại:**

  +Đệ quy tuyến tính.

  +Đệ quy tương hỗ.

  +Đệ quy phi tuyến.

Vấn đề

Một vấn đề mang tính đệ quy nếu như nó có thể được giải quyết thông qua kết quả của chính vấn đề đó nhưng với đầu vào đơn giản hơn.

Ví dụ:

$$n! = 1*2*3*\ldots*n$$

$$(n-1)! = 1*2*3*\ldots*(n-1)$$

$$\Rightarrow n! = n*(n-1)!$$

Trường hợp cơ bản

Trường hợp cơ bản là một input đủ nhỏ để ta có thể giải quyết vấn đề mà không cần lời gọi đệ quy.

Ví dụ:

$n! = 1*2*3*\ldots*n$

$n! = n*(n-1)!$

$1! = 1*(1-1)!$ → $0! = 1$ (trường hợp cơ bản)

$S(n) = 1+2+3+\ldots+n$

$S(n) = n+S(n-1)$

$S(2) = 2+S(2-1)$ → $S(1)=1$ (trường hợp cơ bản)

Đệ quy tuyến tính

Hàm đệ quy tuyến tính chỉ có duy nhất một lần gọi lại chính nó

```
int giai_thua(int n)
{
    if (n == 0) return 1;
    else return n * giai_thua(n - 1);
}
```

Ngay cả khi lời gọi đệ quy xuất hiện nhiều lần nhưng chỉ một lần được chạy

```
int uscln(int a, int b)
{
    if (a == b) return a;
    else if (a > b) return uscln(a - b, b);
    else uscln(a, b - a);
}
```

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Đệ quy phi tuyến

Hàm đệ quy phi tuyến nhiều hơn một lần gọi lại chính nó

```
int fibonacci(int n)
{
    if (n < 2) return 1;
    else return fibonacci(n - 1) + fibonacci(n  - 2);
}
```

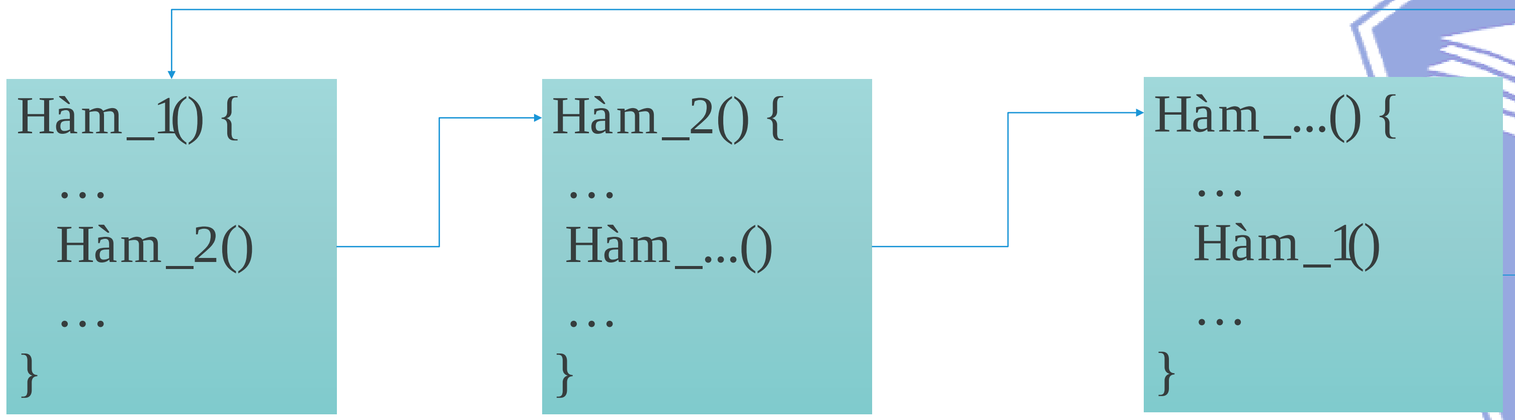
Hàm Fibonacci gọi lại chính nó 02 lần, trường hợp đặc biệt của đệ quy phi tuyến: **đệ quy nhị phân**

Đoạn code trên khó chuyển sang cấu trúc lặp

ĐOÀN KHOA CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP
Sharing is learning

Đệ quy tương hỗ

- Đệ quy hỗ tương – mutual recursion. Còn gọi đệ quy gián tiếp – indirect recursion.
- Hàm không trực tiếp gọi lại chính nó mà gọi thông qua một (hoặc nhiều) hàm khác



Đề thi cuối kì I 2018-2019

Câu 1: Cho đoạn chương trình sau: (1 điểm)

```
#include <iostream>
using namespace std;
int main()
{
    int x=3;
    int &y = x;
    int *p = &x;
    x=2019;
    return 0;
}
```

Khi chương trình thực thi, biến x được cấp phát bộ nhớ có địa chỉ 0x28fefc (địa chỉ biến x), biến p được cấp phát bộ nhớ có địa chỉ 0x28fef8 (địa chỉ biến p). Hãy cho biết khi chương trình thực hiện đến lệnh return 0.

Giá trị của p là :..... Giá trị của &p là:.....

Giá trị của *p là:..... Giá trị của y là:.....

Đề thi cuối kì I 2018-2019

Câu 2: Cho đoạn chương trình sau:

```
#include <iostream>
using namespace std;
int main()
{
    int a[5];
    int *p = a;
    int x=2;
    for(int i=0 ; i<5; i++)  a[i] = i;
    //  lệnh sẽ được thêm
    return 0 ;
}
```

Xác định những lệnh nào sau đây là hợp lệ và không hợp lệ khi thêm vào trong đoạn chương trình trên. Nếu không hợp lệ, giải thích vì sao?

Đề thi cuối kì I 2018-2019

Câu 2: Cho đoạn chương trình sau:

```
#include <iostream>
using namespace std;
int main()
{
    int a[5];
    int *p = a;
    int x=2;
    for(int i=0; i<5; i++)  a[i] = i;
    //  lệnh sẽ được thêm
    return 0;
}
```

a) p=a[0];
b) p[1]=x-2;
c) *(p+2)=x;
d) (*p)+2=x;
e) p=&a[2];
f) a++;

Đề thi cuối kì I 2018-2019

Câu 3: Hãy viết hàm DemSoKyTuNguyenAm có chức năng đếm số ký tự là ký tự nguyên âm (‘a’, ‘e’, ‘o’, ‘i’, ‘u’) có trong chuỗi ký tự.

Lưu ý: các ký tự được nhập không có dấu, số lượng ký tự không quá 256. Ví dụ: - chuỗi ký tự "Hello UIT" có 4 ký tự nguyên âm. Chuỗi ký tự "Truong Dai hoc Cong nghe Thong tin " có 9 ký tự nguyên âm.

```
#include<iostream>
#include<cstring>using namespace std;
//Viết hàm DemSoKyTuNguyenAm để hoàn chỉnh chương trình.
```

Đề thi cuối kì I 2018-2019

```
int DemSoKyTuNguyenAm(char *s)
{
    int n = strlen(s);
    int dem=0;
    for(int i =0; i<n; i++)
        if(s[i]=='A'||s[i]=='E'||s[i]=='O'||s[i]=='I'||s[i]=='U')
dem++;
    for(int i =0; i<n; i++)
        if(s[i]=='a'||s[i]=='e'||s[i]=='o'||s[i]=='i'||s[i]=='u')
dem++;
    return dem;
}
```

Đề thi cuối kì I 2018-2019

Câu 4: Hãy viết các hàm theo yêu cầu như sau:

a) Hàm nhập mảng số nguyên a có n phần tử từ bàn phím.

b) Tìm vị trí số lớn nhất đầu tiên trong mảng.

c) In các số nguyên tố có trong mảng số nguyên ra màn hình.

d) Kiểm tra tính chẵn lẻ của mảng.

- Hàm sẽ trả về giá trị 1 nếu mảng chứa toàn số chẵn

- Hàm sẽ trả về giá trị -1 nếu mảng chứa toàn số lẻ

- Hàm sẽ trả về giá trị 0 nếu mảng không chứa toàn số chẵn hay không chứa toàn số lẻ ?

e) Viết hàm main để gọi thực thi 4 hàm đã viết trong câu a,b,c,d ở trên.

Đề thi cuối kì I 2018-2019

Câu 5: a) Hãy định nghĩa kiểu dữ liệu Point biểu diễn một điểm trong mặt phẳng Oxy.

b) Viết hàm CmpDis nhận vào 2 tham số là điểm (kiểu Point):

- Hàm trả về giá trị -1 nếu khoảng cách từ gốc toạ độ đến điểm thứ nhất NHỎ HƠN khoảng cách từ gốc toạ độ đến điểm thứ hai.
- Hàm trả về giá trị 1 nếu khoảng cách từ gốc toạ độ đến điểm thứ nhất LỚN HƠN khoảng cách từ gốc toạ độ đến điểm thứ hai.
- Hàm trả về giá trị 0 nếu khoảng cách từ gốc toạ độ đến điểm thứ nhất BẰNG khoảng cách từ gốc toạ độ đến điểm thứ hai.

Đề thi cuối kì I 2018-2019

```
struct Point
{    int x,y;
};
int CmpDis(Point A, Point B)
{
    int dA = A.x*A.x + A.y*A.y;
    int dB = B.x*B.x + B.y*B.y;
    if (dA<dB) return -1;
    if (dA>dB) return 1;
    return 0;
}
```

Đề thi cuối kì I 2018-2019

Câu 6: Cho mảng 2 chiều a có r dòng và c cột. Các giá trị trong mảng được cho ở bảng.

Hãy cho biết các giá tri trong mảng sau khi thực hiện đoạn chương trình sau:

```
for(int i = 0 ; i <r; i++)
    { for(int j = 0 ; j <c/2; j++)
        {
            swap(a[i][j], a[i][c-1-j]);
        }
    }
```

Trong đó swap là hàm chuẩn trong thư viện cstdlib, có chức năng đổi chổ giá trị của hai tham số.

1	2	3	4	5
6	7	8	9	8
7	6	5	4	3
2	1	2	3	4

Đề thi cuối kì I 2018-2019

5	4	3	2	1
8	9	8	7	6
3	4	5	6	7
4	3	2	1	2

BAN HỌC TẬP KHOA CÔNG NGHỆ PHẦN MỀM
CHUỖI TRAINING CUỐI HỌC KÌ 1 NĂM HỌC 2019 - 2020

ĐOÀN KHOA
CÔNG NGHỆ PHẦN MỀM
BAN HỌC TẬP

Sharing is learning

HẾT

CẢM ƠN CÁC BẠN ĐÃ THEO DÕI.
CHÚC CÁC BẠN CÓ KẾT QUẢ THI THẬT TỐT!

Ban học tập
Khoa Công Nghệ Phần Mềm
Trường ĐH Công Nghệ Thông Tin
ĐHQG Hồ Chí Minh

Email / Group
bht.cnpm.uit@gmail.com
fb.com/groups/bht.cnpm.uit